

Java Threads

[< Previous](#)[Next >](#)

Java Threads

Threads allows a program to operate more efficiently by doing multiple things at the same time.

Threads can be used to perform complicated tasks in the background without interrupting the main program.

Creating a Thread

There are two ways to create a thread.

It can be created by extending the `Thread` class and overriding its `run()` method:

Extend Syntax

[Get your own Java Server](#)

```
public class Main extends Thread {
    public void run() {
        System.out.println("This code is running in a thread");
    }
}
```

Another way to create a thread is to implement the `Runnable` interface:

Implement Syntax

```
public class Main implements Runnable {
    public void run() {
```

```
        System.out.println("This code is running in a thread");
    }
}
```

Running Threads

If the class extends the `Thread` class, the thread can be run by creating an instance of the class and call its `start()` method:

Extend Example

```
public class Main extends Thread {
    public static void main(String[] args) {
        Main thread = new Main();
        thread.start();
        System.out.println("This code is outside of the thread");
    }
    public void run() {
        System.out.println("This code is running in a thread");
    }
}
```

[Try it Yourself »](#)

If the class implements the `Runnable` interface, the thread can be run by passing an instance of the class to a `Thread` object's constructor and then calling the thread's `start()` method:

Implement Example

```
public class Main implements Runnable {
    public static void main(String[] args) {
        Main obj = new Main();
        Thread thread = new Thread(obj);
        thread.start();
        System.out.println("This code is outside of the thread");
    }
}
```

```
public void run() {  
    System.out.println("This code is running in a thread");  
}  
}
```

Try it Yourself »

Differences between "extending" and "implementing" Threads

The major difference is that when a class extends the Thread class, you cannot extend any other class, but by implementing the Runnable interface, it is possible to extend from another class as well, like: class `MyClass extends OtherClass implements Runnable`.

Concurrency Problems

Because threads run at the same time as other parts of the program, there is no way to know in which order the code will run. When the threads and main program are reading and writing the same variables, the values are unpredictable. The problems that result from this are called concurrency problems.

Example

A code example where the value of the variable **amount** is unpredictable:

```
public class Main extends Thread {  
    public static int amount = 0;  
  
    public static void main(String[] args) {  
        Main thread = new Main();  
        thread.start();  
        System.out.println(amount);  
        amount++;  
        System.out.println(amount);  
    }  
  
    public void run() {  
        amount++;  
    }  
}
```

```
}  
}
```

Try it Yourself »

To avoid concurrency problems, it is best to share as few attributes between threads as possible. If attributes need to be shared, one possible solution is to use the `isAlive()` method of the thread to check whether the thread has finished running before using any attributes that the thread can change.

Example

Use `isAlive()` to prevent concurrency problems:

```
public class Main extends Thread {  
    public static int amount = 0;  
  
    public static void main(String[] args) {  
        Main thread = new Main();  
        thread.start();  
        // Wait for the thread to finish  
        while(thread.isAlive()) {  
            System.out.println("Waiting...");  
        }  
        // Update amount and print its value  
        System.out.println("Main: " + amount);  
        amount++;  
        System.out.println("Main: " + amount);  
    }  
    public void run() {  
        amount++;  
    }  
}
```

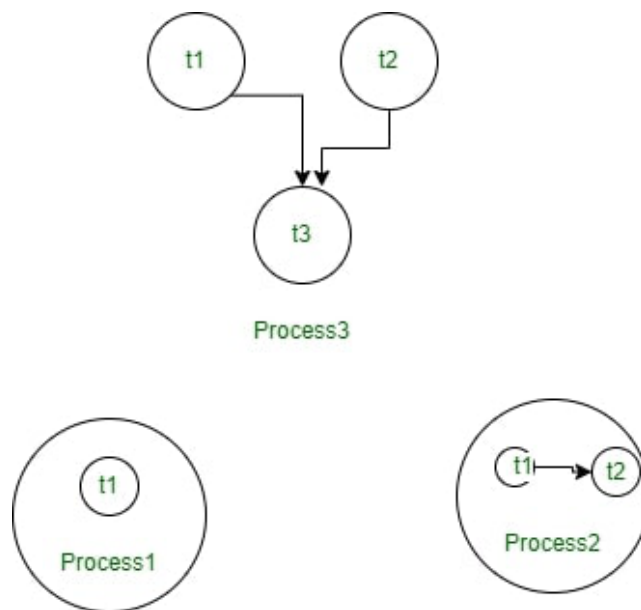
Try it Yourself »

Java Threads

Last Updated : 13 Aug, 2024



Typically, we can define threads as a subprocess with lightweight with the smallest unit of processes and also has separate paths of execution. The main advantage of multiple threads is efficiency (allowing multiple things at the same time). For example, in MS Word. one thread automatically formats the document while another thread is taking user input. Another advantage is quick response, if we use multiple threads in a process and if a thread gets stuck due to lack of resources or an exception, the other threads can continue to execution, allowing the process (which represents an application) to continue to be responsive.



Threads in a Shared Memory Environment in OS

As we can observe in, the above diagram a thread runs inside the process and there will be context-based switching between threads there can be multiple processes running in OS, and each process again can have multiple threads running simultaneously. The Multithreading concept is popularly applied in games, animation...etc.

The Concept Of Multitasking

To help users Operating System accommodates users the privilege of multitasking, where users can perform multiple actions simultaneously on the machine. This

Multitasking can be enabled in two ways:

1. **Process-Based Multitasking**
2. **Thread-Based Multitasking**

1. Process-Based Multitasking (Multiprocessing)

In this type of Multitasking, processes are heavyweight and each process was allocated by a separate memory area. And as the process is heavyweight the cost of communication between processes is high and it takes a long time for switching between processes as it involves actions such as loading, saving in registers, updating maps, lists, etc.

2. Thread-Based Multitasking

As we discussed above Threads are provided with lightweight nature and share the same address space, and the cost of communication between threads is also low.

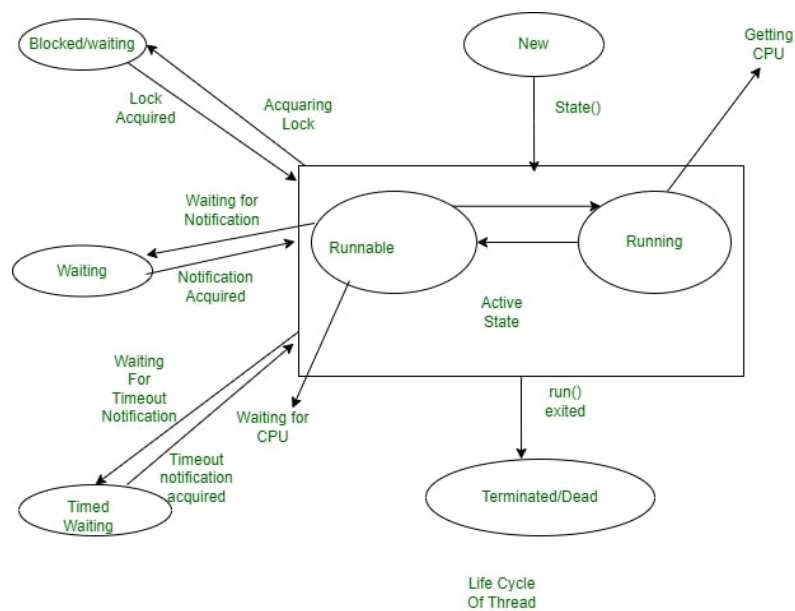
Why Threads are used?

Now, we can understand why threads are being used as they had the advantage of being lightweight and can provide communication between multiple threads at a Low Cost contributing to effective multi-tasking within a shared memory environment.

Life Cycle Of Thread

There are different states Thread transfers into during its lifetime, let us know about those states in the following lines: in its lifetime, a thread undergoes the following states, namely:

1. New State
2. Active State
3. Waiting/Blocked State
4. Timed Waiting State
5. Terminated State



We can see the working of different states in a Thread in the above Diagram, let us know in detail each and every state:

1. New State

By default, a Thread will be in a new state, in this state, code has not yet been run and the execution process is not yet initiated.

2. Active State

A Thread that is a new state by default gets transferred to Active state when it invokes the start() method, his Active state contains two sub-states namely:

- **Runnable State:** In This State, The Thread is ready to run at any given time and it's the job of the Thread Scheduler to provide the thread time for the runnable state preserved threads. A program that has obtained Multithreading shares slices of time intervals which are shared between threads hence, these threads run for some short span of time and wait in the runnable state to get their schedules slice of a time interval.
- **Running State:** When The Thread Receives CPU allocated by Thread Scheduler, it transfers from the “Runnable” state to the “Running” state. and after the expiry of its given time slice session, it again moves back to the “Runnable” state and waits for its next time slice.

3. Waiting/Blocked State

If a Thread is inactive but on a temporary time, then either it is a waiting or blocked state, for example, if there are two threads, T1 and T2 where T1 needs to communicate to the camera and the other thread T2 already using a camera to scan then T1 waits until T2 Thread completes its work, at this state T1 is parked in waiting for the state, and in another scenario, the user called two Threads T2 and T3 with the same functionality and both had same time slice given by Thread Scheduler then both Threads T1, T2 is in a blocked state. When there are multiple threads parked in a Blocked/Waiting state Thread Scheduler clears Queue by rejecting unwanted Threads and allocating CPU on a priority basis.

4. Timed Waiting State

Sometimes the longer duration of waiting for threads causes starvation, if we take an example like there are two threads T1, T2 waiting for CPU and T1 is undergoing a Critical Coding operation and if it does not exist the CPU until its operation gets executed then T2 will be exposed to longer waiting with undetermined certainty, In order to avoid this starvation situation, we had Timed Waiting for the state to avoid that kind of scenario as in Timed Waiting, each thread has a time period for which `sleep()` method is invoked and after the time expires the Threads starts executing its task.

5. Terminated State

A thread will be in Terminated State, due to the below reasons:

- Termination is achieved by a Thread when it finishes its task Normally.
- Sometimes Threads may be terminated due to unusual events like segmentation faults, exceptions...etc. and such kind of Termination can be called Abnormal Termination.
- A terminated Thread means it is dead and no longer available.

What is Main Thread?

As we are familiar, we create Main Method in each and every Java Program, which acts as an entry point for the code to get executed by JVM, Similarly in this Multithreading Concept, Each Program has one Main Thread which was provided

by default by JVM, hence whenever a program is being created in java, JVM provides the Main Thread for its Execution.

How to Create Threads using Java Programming Language?

We can create Threads in java using two ways, namely :

1. Extending Thread Class
2. Implementing a Runnable interface

1. By Extending Thread Class

We can run Threads in Java by using Thread Class, which provides constructors and methods for creating and performing operations on a Thread, which extends a Thread class that can implement Runnable Interface. We use the following constructors for creating the Thread:

- Thread
- Thread(Runnable r)
- Thread(String name)
- Thread(Runnable r, String name)

Sample code to create Threads by Extending Thread Class:

Java

```
1  import java.io.*;
2  import java.util.*;
3
4  public class GFG extends Thread {
5      // initiated run method for Thread
6      public void run()
7      {
8          System.out.println("Thread Started
9          Running...");
10     }
11     public static void main(String[] args)
12     {
13         GFG g1 = new GFG();
```

```
13
14         // Invoking Thread using start() method
15         g1.start();
16     }
17 }
```

Output

Thread Started Running...

Sample code to create Thread by using Runnable Interface:

Java

```
1  import java.io.*;
2  import java.util.*;
3
4  public class GFG implements Runnable {
5      // method to start Thread
6      public void run()
7      {
8          System.out.println(
9              "Thread is Running Successfully");
10     }
11
12     public static void main(String[] args)
13     {
14         GFG g1 = new GFG();
15         // initializing Thread Object
16         Thread t1 = new Thread(g1);
17         t1.start();
18     }
19 }
```

Output

Thread is Running Successfully

Sample Code to create Thread in Java using Thread(String name):

Java

```
1  import java.io.*;
2  import java.util.*;
3
4  public class GFG {
5      public static void main(String args[])
6      {
7          // Thread object created
8          // and initiated with data
9          Thread t = new Thread("Hello Geeks!");
10
11         // Thread gets started
12         t.start();
13
14         // getting data of
15         // Thread through String
16         String s = t.getName();
17         System.out.println(s);
18     }
19 }
```

Output

Hello Geeks!

Sample Java Code which creates Thread Object by using Thread(Runnable r, String name):

Java

```
1  import java.io.*;
2  import java.util.*;
3
4  public class GFG implements Runnable {
5      public void run()
```

```

6      {
7          System.out.println(
8              "Thread is created and running
successfully...");
9      }
10     public static void main(String[] args)
11     {
12         // aligning GFG Class with
13         // Runnable interface
14         Runnable r1 = new GFG();
15         Thread t1 = new Thread(r1, "My Thread");
16         // Thread object started
17         t1.start();
18         // getting the Thread
19         // with String Method
20         String str = t1.getName();
21         System.out.println(str);
22     }
23 }

```

Output

```

My Thread
Thread is created and running successfully...

```

Java Program to explore different Thread States:

Let us see the working of thread states by implementing them on Threads t1 and t2.

Output:

Java

```

1  import java.io.*;
2  import java.util.*;
3
4  class GFG implements Runnable {
5      public void run()
6      {

```

```

7      // implementing try-catch Block to set sleep
state
8      // for inactive thread
9      try {
10         Thread.sleep(102);
11     }
12     catch (InterruptedException i1) {
13         i1.printStackTrace();
14     }
15     System.out.println(
16         "The state for t1 after it invoked join
method() on thread t2"
17         + " " + ThreadState.t1.getState());
18
19     // implementing try-catch block
20     try {
21         Thread.sleep(202);
22     }
23     catch (InterruptedException i2) {
24         i2.printStackTrace();
25     }
26 }
27 }
28
29 // creation of ThreadState class
30 // to implement Runnable interface
31 public class ThreadState implements Runnable {
32     public static Thread t1;
33     public static ThreadState o1;
34     public static void main(String args[])
35     {
36         o1 = new ThreadState();
37         t1 = new Thread(o1);
38         System.out.println("post-spanning, state of t1
is"
39                             + " " + t1.getState());
40         // lets invoke start() method on t1
41         t1.start();
42         // Now, Thread t1 is moved to runnable state
43         System.out.println(
44             "post invoking of start() method, state of
t1 is"

```

```

45         + " " + t1.getState());
46     }
47     public void run()
48     {
49         GFG g1 = new GFG();
50         Thread t2 = new Thread(g1);
51         // Thread is created and its in new state.
52         t2.start();
53         // Now t2 is moved to runnable state
54         System.out.println(
55             "state of t2 Thread, post-calling of
56             start() method is"
57             + " " + t2.getState());
58         // create a try-catch block to set t1 in
59         waiting
60         // state
61         try {
62             Thread.sleep(202);
63         }
64         catch (InterruptedException i2) {
65             i2.printStackTrace();
66         }
67         System.out.println(
68             "State of Thread t2 after invoking to
69             method sleep() is"
70             + " " + t2.getState());
71         try {
72             t2.join();
73             System.out.println(
74                 "State of Thread t2 after join() is"
75                 + " " + t2.getState());
76         }
77         catch (InterruptedException i3) {
78             i3.printStackTrace();
79         }
80         System.out.println(
81             "state of Thread t1 after completing the
82             execution is"
83             + " " + t1.getState());
84     }
85 }

```

Output

```
post-spanning, state of t1 is NEW
post invoking of start() method, state of t1 is RUNNABLE
state of t2 Thread, post-calling of start() method is RUNNABLE
The state for t1 after it invoked join method() on thread t2
TIMED_WAITING
State of Thread t2 after invoking to method sleep() is TIMED_WAITING
State of Thread t2 after join() is TERMINATED
state of Thread t1 after completing the execution is RUNNABLE
```

 Comment

More info 

Next Article 

Java.lang.Thread Class in Java

Similar Reads

Java Tutorial

Java is one of the most popular and widely used programming languages which is used to develop mobile apps, desktop apps, web apps, web servers, games, and enterprise-level systems. Java was...

 4 min read

Java Overview



Java Basics



Java Flow Control



Java Methods



Java Arrays



Java Strings 

Java OOPs Concepts 

Java Interfaces 

Java Collections 

Java Exception Handling 

Java Multithreading 

Java Multithreading Tutorial

Threads are the backbone of multithreading. We are living in a real world which in itself is caught on the web surrounded by lots of applications. With the advancement in technologies, we cannot achieve the...

 15+ min read

Java Threads

Typically, we can define threads as a subprocess with lightweight with the smallest unit of processes and also has separate paths of execution. The main advantage of multiple threads is efficiency (allowin...

 9 min read

Java.lang.Thread Class in Java

Thread is a line of execution within a program. Each program can have multiple associated threads. Each thread has a priority which is used by the thread scheduler to determine which thread must run first....

 8 min read

Runnable interface in Java

java.lang.Runnable is an interface that is to be implemented by a class whose instances are intended to be executed by a thread. There are two ways to start a new Thread - Subclass Thread and implement...

 3 min read

Lifecycle and States of a Thread in Java

A thread in Java at any point of time exists in any one of the following states. A thread lies only in one of the shown states at any instant: New StateRunnable StateBlocked StateWaiting StateTimed Waiting...

🕒 6 min read

Main thread in Java

Java provides built-in support for multithreaded programming. A multi-threaded program contains two or more parts that can run concurrently. Each part of such a program is called a thread, and each thread...

🕒 4 min read

Java Thread Priority in Multithreading

As we already know java being completely object-oriented works within a multithreading environment in which thread scheduler assigns the processor to a thread based on the priority of thread. Whenever...

🕒 5 min read

Naming a thread and fetching name of current thread in Java

Thread can be referred to as a lightweight process. Thread uses fewer resources to create and exist in the process; thread shares process resources. The main thread of Java is the thread that is started whe...

🕒 5 min read

Difference between Thread.start() and Thread.run() in Java

In Java's multi-threading concept, start() and run() are the two most important methods. Below are some of the differences between the Thread.start() and Thread.run() methods: New Thread creation:...

🕒 3 min read

Thread.sleep() Method in Java With Examples

Thread Class is a class that is basically a thread of execution of the programs. It is present in Java.lang package. Thread class contains the Sleep() method. There are two overloaded methods of Sleep()...

🕒 4 min read

Daemon Thread in Java

In Java, daemon threads are low-priority threads that run in the background to perform tasks such as garbage collection or provide services to user threads. The life of a daemon thread depends on the...

🕒 4 min read

Thread Safety and how to achieve it in Java

As we know Java has a feature, Multithreading, which is a process of running multiple threads simultaneously. When multiple threads are working on the same data, and the value of our data is...

🕒 5 min read

Thread Pools in Java